目录

1.设计需求与目标分析 1

2.设计思路 1

3.Cordic 算法详细设计 2

4.IP 核设计与项目框架搭建 6

5.PS 端代码编写与调试 11

6.成果演示与效果评估 13

7.项目中的困难与解决方案 18

8.项目改进与创新 19

9.项目总结 20

10.参考文献 21

1. 设计需求与目标

1.1 设计需求

（1）用 Verilog 实现 Cordic 算法；

（2）能够计算 sin、cos、arcsin、arctan 等函数；

（3）验证输入通过 PS 的串口送到 PL 的用户 Cordic 核；

（4）计算结果通过 PS 端的串口输出；

1.2 设计目标

（1）确保能够准确计算 sin、cos、arcsin、arctan 等基础三角与反三角函数。

（2）坚持简洁而结构化的代码设计原则，采用参数化和模块化设计方法，避免冗余代码。同时，在支持 4 种函数计算的复杂逻辑下，优化有限状态机与数据通路，在 FPGA 资源消耗与计算精度之间寻找最佳平衡点。

2. 设计思路

首先，构建基于 AXI Uartlite 的 PS-PL 异构协同架构。本设计基于 Zynq-7000 SoC 平台，PS 端作为主控制器负责数据交互与指令调度，PL 端作为协处理器负责高强度计算。两者通过 AXI Uartlite IP 核建立通信桥梁：PS 端将待测数据与操作指令通过 AXI 总线发送，经由 Uartlite 转换为串行数据流，PL 端自定义接收模块解析数据帧并送入计算核心，最后将结果回传，实现了低资源的软硬协同交互。

其次，设计高效的 Cordic 迭代计算引擎。算法实现摒弃了传统查表法资源消耗大的缺点，基于统一的旋转迭代逻辑构建核心架构。通过 16 级流水线设计与 Q16.16 定点化策略，仅需复用同一套移位器与加法器资源，配合旋转与向量两种模式的动态切换，即可高效支持 sin、cos、arctan 的计算；此外，针对 arcsin 函数，创新性地提出了基于无开方逼近的特殊算法，在保证精度的前提下显著降低了硬件复杂度，资源利用率高。

最后，采用状态机解析通信协议与硬件在环验证。在数据链路层，PL 端自定义的 UART 接收模块利用有限状态机（FSM）实现数据帧的同步、拼接与指令解

析，确保了通信的稳定性；在计算核心层，则利用全流水线架构实现数据的高速并行吞吐。系统验证采用硬件在环方案，PS 端通过串口与上位机交互，将用户输入实时下发至 PL 核并回传计算结果，形成了完整的闭环验证系统。

3. Cordic 算法详细设计

3.1 通用 Cordic 方程

对于 Cordic 中基础函数的实现，可以由如下图通用公式得到：

$$\begin{cases} x_{i+1} = x_i - \mu \cdot d_i \cdot (y_i \cdot 2^{-i}) \\ y_{i+1} = y_i + d_i \cdot (x_i \cdot 2^{-i}) \\ z_{i+1} = z_i - d_i \cdot e_i \end{cases}$$

该方程通过参数 μ 切换三种工作模式，对应的基本旋转角度 e_i 定义如下：

$$e_i = \begin{cases} \arctan(2^{-i}) & (\mu = 1, Circular) \\ 2^{-i} & (\mu = 0, Linear) \\ arctanh(2^{-i}) & (\mu = -1, Hyperbolic) \end{cases}$$

在迭代过程中，旋转方向 d_i 的判定取决于算法的工作目标，即旋转模式或向量模式：

$$d_i = \begin{cases} sign(z_i) & (z \to 0) \\ -sign(x_i y_i) & (y \to 0) \end{cases}$$

由于伪旋转会导致向量模长发生伸缩，故而硬件输出时需乘以校正因子 1/K 以还原真实值 N 次迭代后的总伸缩因子 K 为：

$$K = \prod_{i=0}^{N-1} \sqrt{1 + \mu 2^{-2i}}$$

基于上述统一迭代方程，Cordic 算法展现了极高的硬件复用性。该算法将复杂的三角函数与双曲函数运算，统一转化为简单的移位（Shift）与加减（Add/Sub）操作，从而避免了在 FPGA 中使用昂贵的硬件乘法器资源。

具体而言，通过灵活配置坐标系参数 μ 与迭代模式，本设计的通用架构理论上可覆盖以下运算功能（本次我们仅仅实现题目所要求的四个函数，实际该框架可以轻松拓展至其他诸多三角、反三角函数）：

1. 圆周旋转模式($\mu = 1$):
- 旋转模式($z \to 0$)：输入角度，计算正弦、余弦及坐标旋转。
- 向量模式($y \to 0$)：输入坐标，计算反正切及向量模长。

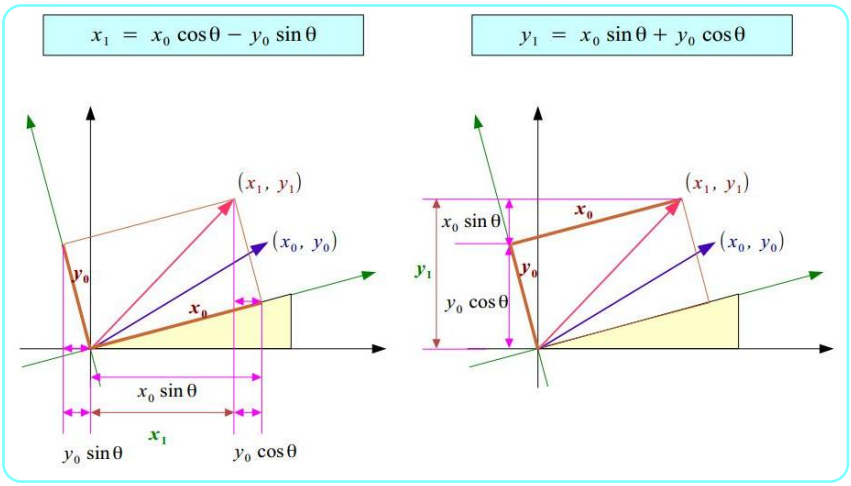
2. 线性旋转模式($\mu = 0$):

- 利用移位加法特性，实现高效的乘法与除法运算。

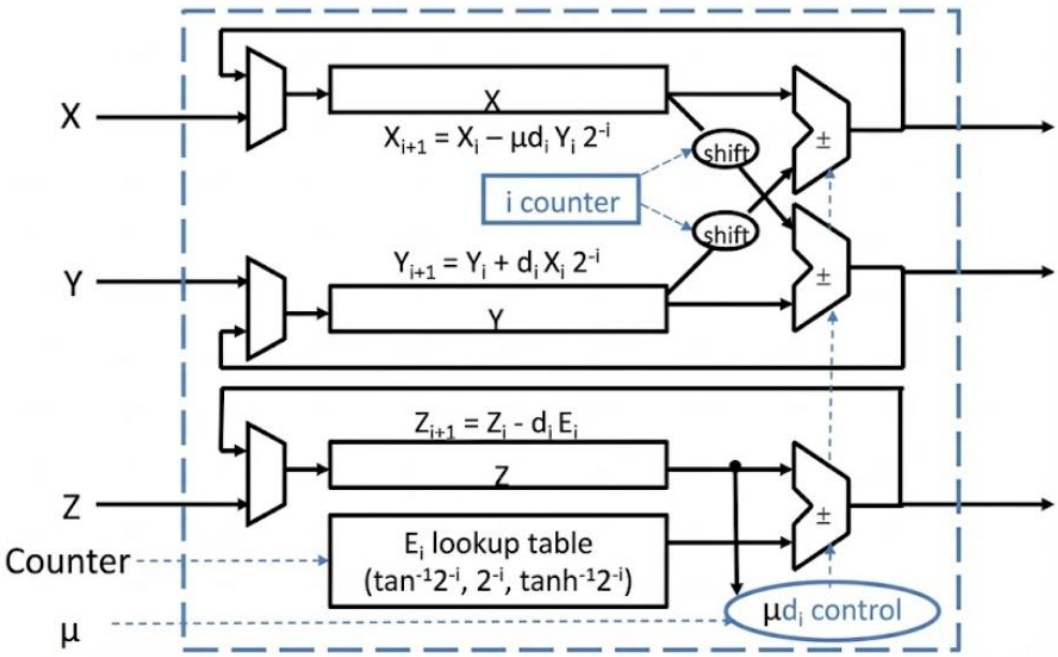
3. 双曲旋转模式 ($\mu = -1$):

- 旋转模式($z \to 0$)：计算双曲正弦、双曲余弦。
- 向量模式($y \to 0$)：计算反双曲正切及自然对数。

下图展示了一个简单的迭代过程：



由此可见，仅需通过有限状态机（FSM）控制输入参数 μ 和查找表 e_i 的切换，即可在同一套数据通路中实现上述所有功能。基于此设计思想，我们构建了如下图所示的通用 Cordic 硬件架构图：



3.2 代码设计

由上图的 Cordic 算法通用模式可以看出，该算法在运算时需要三个初始输入量，分别为 x、y 和 z。同时，为了实现不同数学函数的计算，需根据目标函数的具体类型选择对应的旋转方向系数 μ 以及预设的旋转角度序列 $e^{(i)}$。最终，算法会输出更新后的 x、y、z 值，根据所求函数的不同，从这三个输出中选取一个作为有效结果。

因此，从用户调用角度来看，Cordic 算法可简化为两个主要输入参数：第一个是数据输入 data，在具体计算类型中，该值会被映射到 x、y 或 z 中的某一个关键变量；第二个是类型选择 type，用于指明当前希望执行的计算种类，例如正弦、余弦、反正切、反正弦等。通过这两项输入，算法内部即可自动完成对应的初始化、迭代与输出选择过程。

首先设立参数表如下：

```
assign angle_array[0]  = 32'sd2949120;
assign angle_array[1]  = 32'sd1740992;
assign angle_array[2]  = 32'sd919872;
assign angle_array[3]  = 32'sd466944;
assign angle_array[4]  = 32'sd234368;
assign angle_array[5]  = 32'sd117312;
assign angle_array[6]  = 32'sd58688;
assign angle_array[7]  = 32'sd29312;
assign angle_array[8]  = 32'sd14656;
assign angle_array[9]  = 32'sd7360;
assign angle_array[10] = 32'sd3648;
assign angle_array[11] = 32'sd1856;
assign angle_array[12] = 32'sd896;
assign angle_array[13] = 32'sd448;
assign angle_array[14] = 32'sd256;
assign angle_array[15] = 32'sd128;
```

在迭代设计上，我们选择了 16 级流水线。这主要基于两点考虑：第一是精度匹配，为了得到极高的计算精度，我们要保留 16 位小数精度，那就对应 16 次迭代计算；第二是架构匹配，16 位整数加 16 位小数刚好凑成 32 位，完美贴合 Zynq 的 AXI 总线宽度，这样数据传输最快，资源利用率也最高。

sin 函数和 cos 函数的迭代过程如下图所示：

```
generate
    for(i = 1;i < PIPELINE+1;i = i + 1)begin:cal_xyz
        always@(posedge clk or negedge rst_n)begin
            if(!rst_n)begin
                x[i]    <=  32'sd0;
                y[i]    <=  32'sd0;
                z[i]    <=  32'sd0;
            end
            else    if(z[i-1][31])begin
                x[i]    <=  x[i-1] + (y[i-1] >>> (i-1));
                y[i]    <=  y[i-1] - (x[i-1] >>> (i-1));
                z[i]    <=  z[i-1] + angle_array[i-1];
            end
            else    begin
                x[i]    <=  x[i-1] - (y[i-1] >>> (i-1));
                y[i]    <=  y[i-1] + (x[i-1] >>> (i-1));
                z[i]    <=  z[i-1] - angle_array[i-1];
            end
        end
    end
endgenerate
```

该模块利用 Cordic 算法的旋转模式工作。其核心原理是将一个初始向量 (K,0)（其中 K 为模长补偿因子，约 0.607）在直角坐标系中进行旋转。系统接收输入的角度，通过 16 级流水线迭代，每一级根据剩余角度的符号决定旋转方向（顺时针或逆时针），不断驱动剩余角度趋近于 0。最终，旋转后的向量 X 轴分量即为 cos 值，Y 轴分量即为 sin 值。代码中预先存入了 $\arctan(2^{-i})$ 的角度查找表，通过移位和加减运算代替了复杂的乘法。

arcsin 函数的迭代过程如下图所示：

```
generate
    for(i = 1;i < PIPELINE + 1;i = i + 1)begin:cal_xyz
        always@(posedge clk or negedge rst_n)begin
            if(!rst_n)begin
                currentX[i]   <=  32'sd0;
                currentY[i]   <=  32'sd0;
                currentZ[i]   <=  32'sd0;
                iDatar[i]     <=  32'sd0;

            end
            else    if(currentY[i-1] < iDatar[i-1])begin
                currentX[i]   <=  currentX[i-1] - (currentY[i-1] >>> (i-1));
                currentY[i]   <=  currentY[i-1] + (currentX[i-1] >>> (i-1));
                currentZ[i]   <=  currentZ[i-1] - angle_array[i-1];
                iDatar[i]     <=  iDatar[i-1];
            end
            else    begin
                currentX[i]   <=  currentX[i-1] + (currentY[i-1] >>> (i-1));
                currentY[i]   <=  currentY[i-1] - (currentX[i-1] >>> (i-1));
                currentZ[i]   <=  currentZ[i-1] + angle_array[i-1];
                iDatar[i]     <=  iDatar[i-1];
            end
        end
    end
endgenerate
```

该模块采用了一种特殊的数值逼近逻辑。不同于标准的 Cordic 模式，它设定初始向量为(K,0) ，输入目标正弦值 D。在迭代过程中，系统实时比较当前向量的 Y 分量与目标值 D 的大小：如果 Y<D，则向正方向旋转以增大 Y；反之则向负方向旋转。通过这种动态调整，强制 Y 分量不断逼近输入值 D，而在此过程中累计旋转的角度总和，即为所求的 arcsinD。这种方法巧妙地避免了开平方运算，极大地节省了硬件资源。

arctan 函数的迭代过程如下图所示：

```
generate
    for(j = 1 ;j < 17 ; j = j + 1)
    begin: loop
        always@(posedge clk or negedge rst_n)
        begin
            if( rst_n == 1'b0)
                cal_delay[j] <= 1'b0;
            else
                cal_delay[j] <= cal_delay[j-1];
        end
    end
endgenerate
```

该模块利用 Cordic 算法的向量模式工作。其核心目的是将直角坐标(X,Y)转换为极坐标(R, θ)。系统接收输入的坐标点(X,Y)，在迭代过程中，根据当前 Y 分量的符号决定旋转方向，强制将向量旋转至 X 轴。当迭代结束时，向量被旋转归位，此时累计旋转过的角度相反数即为原向量的角度 arctan(Y/X)。此外，该代码还通过移位近似公式计算了向量的模长。

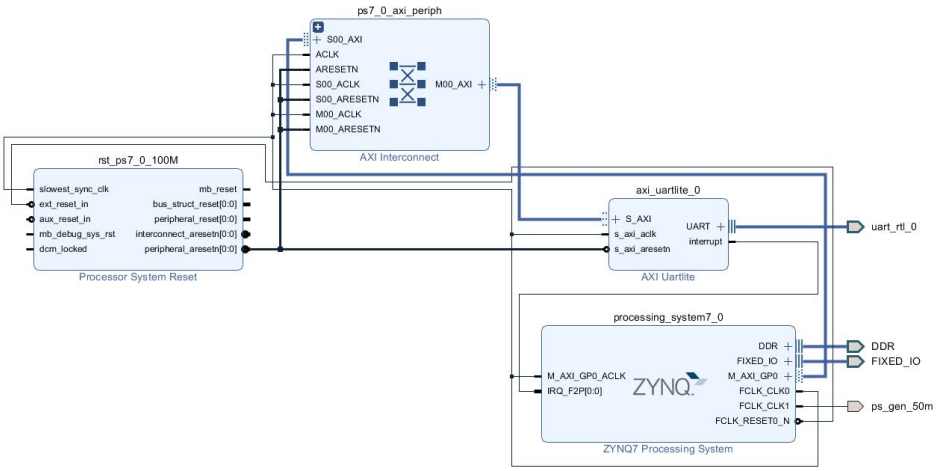
4. IP 核设计与项目框架搭建

项目的整体思路构建于 PS 端与 PL 端的异构协同之上，两者通过 AXI 总线进行高效交互。PS 端作为主控单元，负责指令下发与数据预处理；PL 端作为计算加速单元，接收数据完成运算后回传结果。

在通信方案的选型上，我们对比了 AXI DMA、BRAM 共享内存与 AXI Uartlite 三种主流方式。考虑到本项目聚焦于单点坐标的高实时性计算，而非大规模数据搬运，DMA 方案显得过于臃肿；而 BRAM 方案虽然吞吐量大，但会占用宝贵的片上 Block RAM 资源。相比之下，AXI Uartlite 方案能够将 AXI 协议转换为轻量级的串行通信

协议，逻辑简洁，极大地优化了片上资源。因此，我们最终确立了以 AXI Uartlite 为核心的低资源交互架构。

整体框架由 PS 处理单元、AXI 互联总线、AXI Uartlite IP 核以及 PL 端自定义逻辑四部分组成。整体数据流向设计如下：PS 端将待测数据与操作指令封装成帧，通过 AXI 总线写入 Uartlite 的发送 FIFO；Uartlite 将其转换为串行比特流发送给 PL 端；PL 端自定义 IP 核接收并解析数据，驱动底层的 Cordic 引擎进行流水线计算；计算完成后，结果再次通过串口协议回传至 PS 端。根据设定的框架搭建好的电路连接图如下：



对于 IP 核的设计，主要结构如下所示：

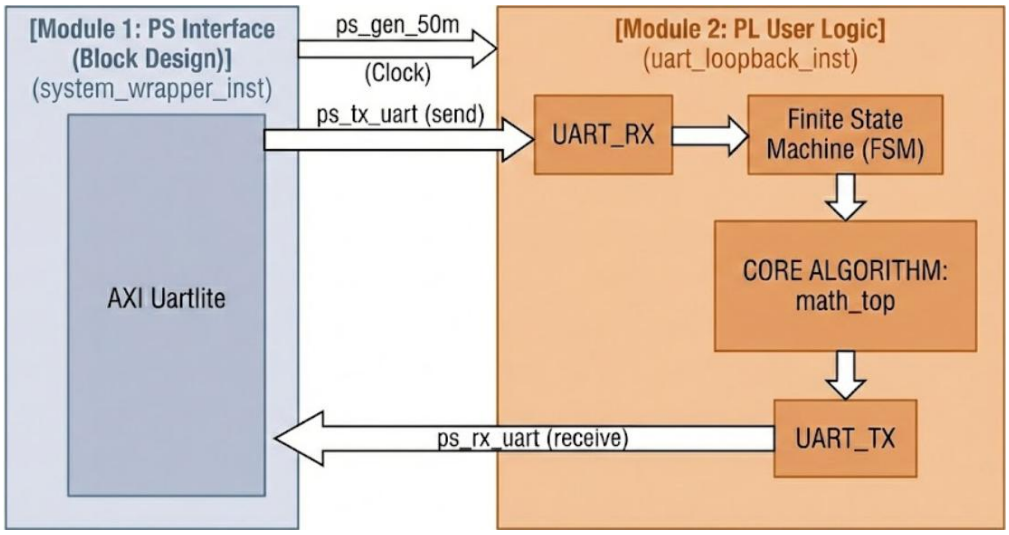
应当注意的是，本系统的硬件交互架构并未局限于 Block Design 的图形化界面内部，而是采用了相对更为底层的 RTL 级互联策略，通过顶层的 Verilog 文件 System_wrapper_top.v 实现了 PS 端接口子系统与 PL 端计算子系统的物理级对接。在这一顶层模块中，我们首先定义了关键的交互线网（Wire）作为通信媒介，包括

用于传输下行控制指令的 ps_tx_uart、传输上行计算结果的 ps_rx_uart，以及由 PS 端 PLL 产生并引出的 50MHz 时钟信号 ps_gen_50m。这种设计将 PL 端自定义逻辑的主时钟直接绑定于 PS 端的输出时钟，从硬件层面确保了软硬协同系统的时钟域同步与时序稳定性。

在具体的下行链路（PS 至 PL）交互中，数据流始于 Block Design 生成的实例 system_wrapper_inst。当 PS 端软件执行发送指令时，AXI Uartlite IP 核将 AXI 总线上的并行数据转换为串行波形，并通过模块的输出端口 uart_rtl_0_txd 发出。该端口在顶层文件中被显式映射至中间线网 ps_tx_uart，随后该线网直接连接至自定义 IP 实例 uart_loopback_inst 的输入端口 uart_rx。这一物理连接意味着，PS 端发出的每一个比特流都会直接驱动 PL 端的状态机进入接收解析状态，从而完成了指令的硬件下发。

对于上行链路（PL 至 PS）的数据回传，系统构建了完整的反馈闭环。当自定义 IP 中的 CORDIC 核心完成流水线计算并拉高有效信号后，状态机将并行结果数据打包，通过模块的输出端口 uart_tx 发送至顶层线网 ps_rx_uart。该线网进一步连接至 PS 端接口实例的接收端口 uart_rtl_0_rxd，将数据写入 AXI Uartlite 的接收 FIFO 中。通过这种清晰的端口映射与信号流转，我们成功构建了一条无需复杂 AXI 互联矩阵即可实现的高效点对点片内互联总线，实现了数据在处理系统与逻辑电路之间的无缝流转。

为了让系统的连接逻辑更加直观，做出如下系统连接逻辑示意图，它从逻辑层级展示了系统的数据流向：



下面展示我们部分核心代码 System_wrapper_top.v：

```
module   System_wrapper_top
  (
      input            i_clk,
      input            i_rst_n,
      // DDR 接口 - inout 类型
      inout wire [14:0] DDR_addr,
      inout wire [2:0]  DDR_ba,
      inout wire        DDR_cas_n,
      inout wire        DDR_ck_n,
      inout wire        DDR_ck_p,
      inout wire        DDR_cke,
      inout wire        DDR_cs_n,
      inout wire [3:0]  DDR_dm,
      inout wire [31:0] DDR_dq,
      inout wire [3:0]  DDR_dqs_n,
      inout wire [3:0]  DDR_dqs_p,
      inout wire        DDR_odt,
      inout wire        DDR_ras_n,
      inout wire        DDR_reset_n,
      inout wire        DDR_we_n,
      // FIXED_IO 接口 - inout 类型
      inout wire        FIXED_IO_ddr_vrn,
      inout wire        FIXED_IO_ddr_vrp,
      inout wire [53:0] FIXED_IO_mio,
      inout wire        FIXED_IO_ps_clk,
      inout wire        FIXED_IO_ps_porb,
      inout wire        FIXED_IO_ps_srstb,
      // UART 接口
      input  wire       uart_rtl_0_rxd,
      output wire       uart_rtl_0_txd
    );
    wire   ps_gen_50m;
    wire   ps_tx_uart;
    wire   ps_rx_uart;

    // 实例化 System_wrapper 模块
    System_wrapper    system_wrapper_inst (
        // DDR 接口
        .DDR_addr          (DDR_addr),
// [14:0]
        .DDR_ba            (DDR_ba),
// [2:0]
        .DDR_cas_n         (DDR_cas_n),
        .DDR_ck_n          (DDR_ck_n),
        .DDR_ck_p          (DDR_ck_p),
        .DDR_cke           (DDR_cke),
        .DDR_cs_n          (DDR_cs_n),
        .DDR_dm            (DDR_dm),
// [3:0]
        .DDR_dq            (DDR_dq),
// [31:0]
        .DDR_dqs_n         (DDR_dqs_n),
// [3:0]
        .DDR_dqs_p         (DDR_dqs_p),
// [3:0]
        .DDR_odt           (DDR_odt),
```

```
        .DDR_ras_n          (DDR_ras_n),
        .DDR_reset_n        (DDR_reset_n),
        .DDR_we_n           (DDR_we_n),
        // FIXED_IO 接口
        .FIXED_IO_ddr_vrn   (FIXED_IO_ddr_vrn),
        .FIXED_IO_ddr_vrp   (FIXED_IO_ddr_vrp),
        .FIXED_IO_mio       (FIXED_IO_mio),
// [53:0]
        .FIXED_IO_ps_clk    (FIXED_IO_ps_clk),
        .FIXED_IO_ps_porb   (FIXED_IO_ps_porb),
        .FIXED_IO_ps_srstb
(FIXED_IO_ps_srstb),
        .ps_gen_50m         (ps_gen_50m),
        // UART 接口
        .uart_rtl_0_rxd     (ps_rx_uart),
        .uart_rtl_0_txd     (ps_tx_uart)
    );
    uart_loopback    uart_loopback_inst
    (
        .Clk     (ps_gen_50m    ),
        .Rst_n   (1'b1          ),
        .uart_rx (ps_tx_uart    ),

        .uart_tx (ps_rx_uart    )
    );
    endmodule
```

通过上述代码，可以看到我们 PS 与 PL 的交互完全由硬件描述语言显式定义，而非依赖工具自动连线。首先，在模块定义的头部，我们声明了三根核心线网（Wire）作为片内的“虚拟导线”：其中 ps_gen_50m 用于将 PS 端产生的时钟引出，作为 PL 端用户逻辑的主时钟，确保了跨模块的时钟同源性；而 ps_tx_uart 与 ps_rx_uart 则构成了下行指令与上行结果的物理通道。随后，在实例化 System_wrapper（即 Block Design 部分）时，我们将 AXI Uartlite 的输出端口.uart_rtl_0_txd 显式连接到了中间线网 ps_tx_uart 上，将其输入端口.uart_rtl_0_rxd 连接到了 ps_rx_uart 上。它证明 PS 端的数据流并没有悬空，而是被正式接入了 FPGA 的逻辑层。代码中占据大量篇幅的 DDR 与 FIXED_IO 接口则是 Zynq 系统的标准配置，用于维持处理器的基本运行。最后，在实例化自定义核心模块 uart_loopback 时，我们直接使用上述同一组线网进行闭环连接。可以看到，逻辑核心的.uart_rx 端口被直接挂载在 ps_tx_uart 线网上，从而能够直接捕获来自 PS 的指令流；同理，核心计算完成后的输出.uart_tx 也直接驱动 ps_rx_uart 回传线路。

下面代码展示了我们的控制逻辑是如何驱动底层的 math_top（Cordic 核心）的，其中结果输出 1、2、3 分别对应正余弦、反正弦和反正切计算结果：

```
math_top    math_top_inst
(
    .i_clk        (Clk          ),
    .i_rst_n      (Rst_n        ),

    .i_valid      (s_input_valid),
    .i_d0         (s_input_data ),

    .o_vaild_1    (o_vaild_1),
    .o_data_sin   (o_data_sin),
    .o_data_cos   (o_data_cos),

    .o_vaild_2    (o_vaild_2),
    .o_data_arcsin(o_data_arcsin),

    .o_vaild_3    (o_vaild_3),
    .o_data_arctg (o_data_arctg)
);
```

针对 IP 核内部结构，设计重点分为通信接口模块与核心计算模块两部分。核心计算模块采用 Verilog 语言实现了基于 16 级流水线的 Cordic 算法。我们摒弃了传统大而全的冗余设计，针对性地优化了圆周旋转与向量/逼近模式，通过复用移位器与加法器资源，实现了 sin、cos、arctan 及 arcsin 这四种关键函数的硬件加速。通信接口模块则负责替代传统的存储器映射接口，直接处理串行数据流，确保指令能够被准确解析并分发给计算核心。

最关键的控制逻辑体现在有限状态机（FSM）的设计上，这直接关系到数据交互的时序稳定性。由于采用了流式通信而非地址读写，我们设计了一套基于数据帧解析的状态流转机制：系统初始处于 IDLE 状态，通过检测起始位“S”来同步数据帧；随后进入接收状态，连续解析十六进制的数据位并重组为 32 位定点数；紧接着读取指令字符（如 A、B、C），根据指令动态配置 Cordic 核的工作模式；最后进入计算等待状态，待流水线输出有效信号后，自动将结果打包回传。这种设计规避了复杂的握手协议，实现了高效、稳定的指令响应闭环。

5. PS 端代码编写与调试

PS 端的代码逻辑主要包含四个核心环节：用户交互与数据采集、数据量化与协议封装、PS-PL 串口通信、以及结果解析与显示。对于输入数据环节，我们利用标准库函数 scanf 实现人机交互。

```
if(Recv_All_Flag) {
    Recv_All_Flag = 0;

    if(Receive_Buffer[0] == 'S') {
        uint32_t hex_value = 0;
        char command = Receive_Buffer[9];
        switch(command) {
            case 'A':
            case 'a':
                AXI_UARTLite_SendData32(&AXI_UART0, 0xaaaaaaaa);
                break;
            case 'B':
            case 'b':
                AXI_UARTLite_SendData32(&AXI_UART0, 0xbbbbbbbb);
                break;
            case 'C':
            case 'c':
                AXI_UARTLite_SendData32(&AXI_UART0, 0xcccccccc);
                break;
            case 'D':
            case 'd':
                AXI_UARTLite_SendData32(&AXI_UART0, 0xdddddddd);
                break;
            default: AXI_UARTLite_SendData32(&AXI_UART0, 0xaaaaaaaa);
            break;

        }
```

对于输入数据的预处理，核心在于将浮点数转换为硬件可处理的定点数格式。根据 Cordic 算法的要求，我们采用 switch case 结构对不同函数进行差异化处理：针对 sin、cos 和 arcsin 模式，采用 Q16.16 格式，将输入浮点数乘以 65536（即左移 16 位）并取整；而针对 arctan 模式，由于需要同时传入 X 和 Y 两个坐标值，我们采用 Q14 格式 分别量化两个分量，并将它们拼接到一个 32 位数据中（高 16 位存 Y，低 16 位存 X），从而实现单次传输双参数，防止溢出并提升传输效率。简单的输出指令见下表：

三角函数	输入值
sin30°	S001E0000A
cos45°	S002D0000B
arcsin(√3/2)	S0000DDB4C
arctan(√3/3)	S24EE4000D
arcsin(-√3/2)	SFFFF224CC
Cos - 45°	SFFD30000B
arctan(-√3/3）	SDB124000D

以 sin30° 为例，输入指令为 S001E0000A，其中 S 代表输入起始，A 代表数据类型（类似的，B 代表余弦，C 代表反正弦，D 代表反正切）。由于输入的是

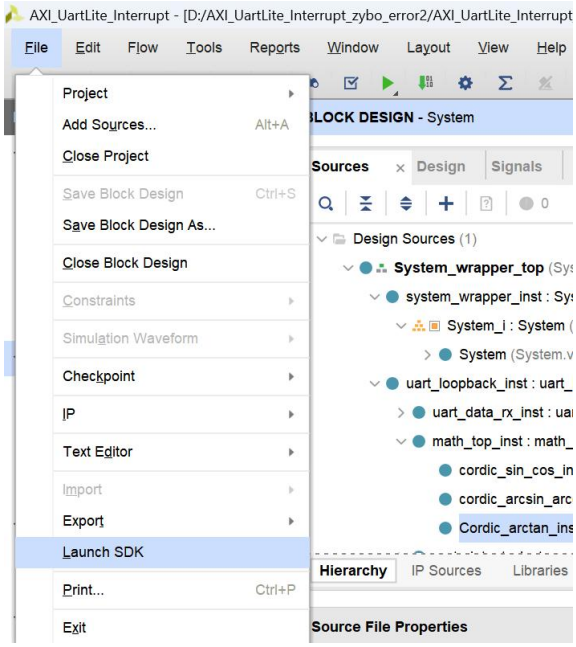
30°，左移 16 位，有 30*(2^16)=1966080，将其转化为 16 进制，为 0x001E0000。故而输入指令 S001E0000A，就代表计算 sin30°。其他函数计算规则相似，都需要先将输入左移，再化为 16 进制。特别地，在处理 arctan 函数时，需要先将输入的分子和分母转化为小于等于 1 的数，再分别左移 14 位并转成 16 进制。例如，输入 arctan(√3/3)后，不妨令 Y=0.577、X=1，有 √3/3=0.577/1，X、Y 分别左移 14 位，有 0.577*(2^14)=9454，1*(2^14)=16384，将其分别转化为 16 进制，为 0x24EE 和 0x4000，故而拼接后的指令为 S24EE4000D。

在完成指令帧的解析与协议封装后，PS 端代码进入核心的通信与后处理阶段。程序首先识别指令后缀，利用 AXI Uartlite 驱动向 PL 端依次下发对应的模式控制字与量化数据载荷，从而启动硬件加速引擎。待 PL 端流水线计算完毕并回传原始数据后，PS 端通过位移操作将其重组为 32 位有符号整型，并执行逆向量化处理——即对于 Q16.16 格式结果，将其除以 65536.0 还原为标准浮点数。最终，代码调用串口发送函数，将硬件计算得到的精确结果实时打印至上位机终端，从而实现从用户指令输入到硬件计算结果回显的完整闭环。

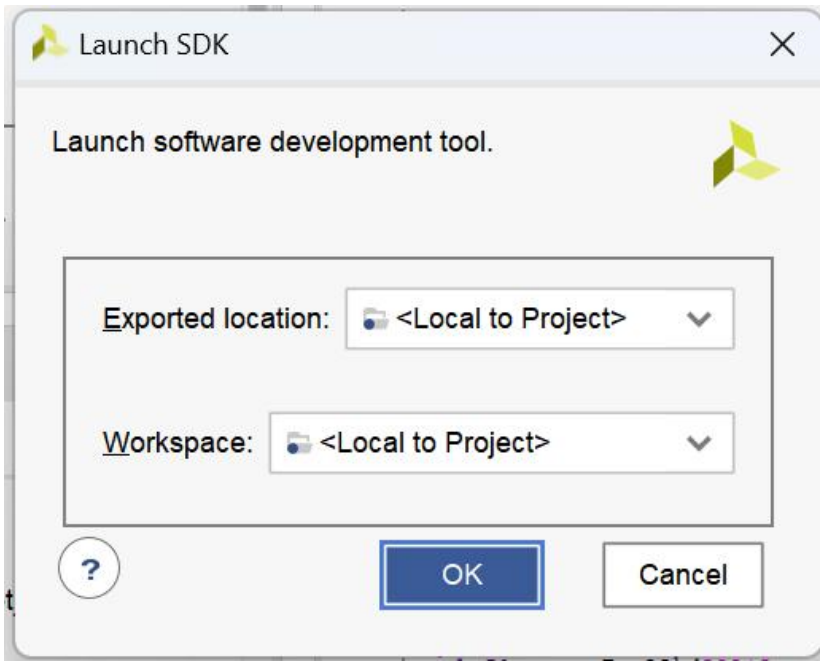
6. 成果演示与效果评估

6.1 操作步骤

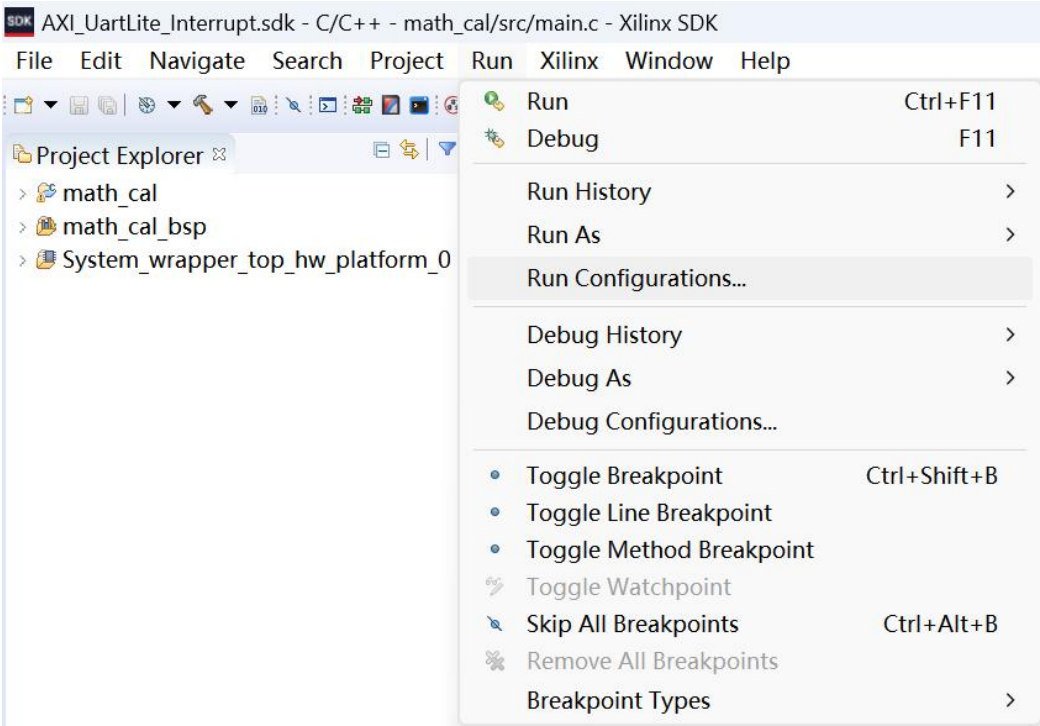
1. 完成所有配置工作后，点击 File——Launch SDK。



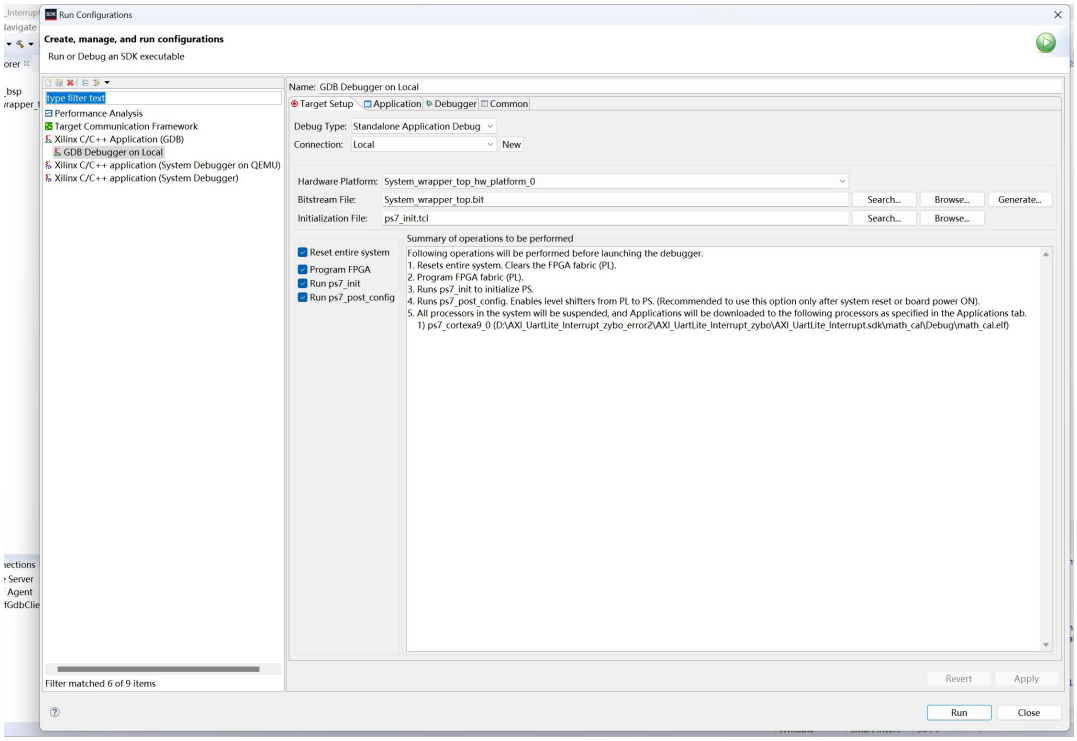
2. 跳出以下界面后，点击 OK。



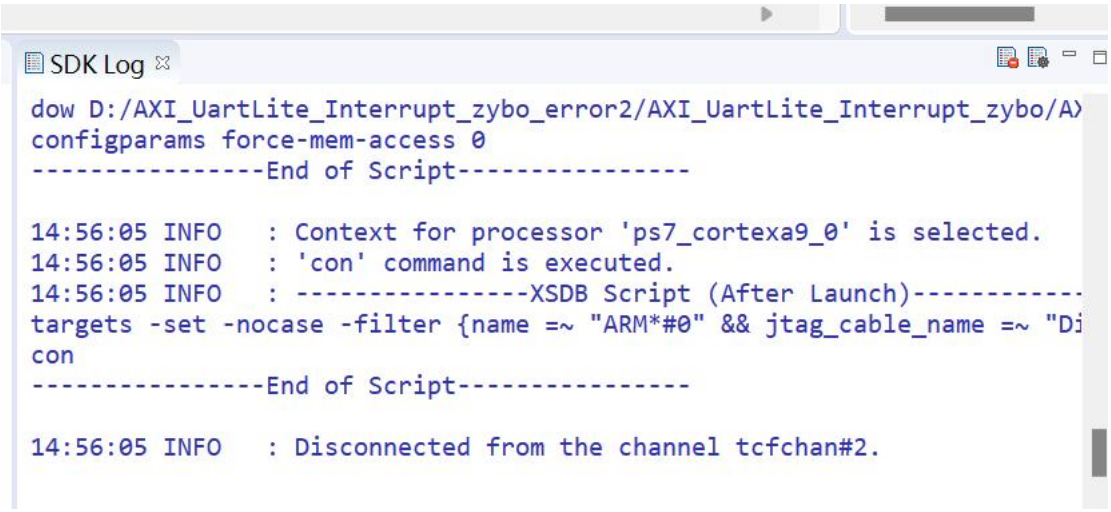
3. 在打开的 SDK 中，点击 Run——Run Configrations。



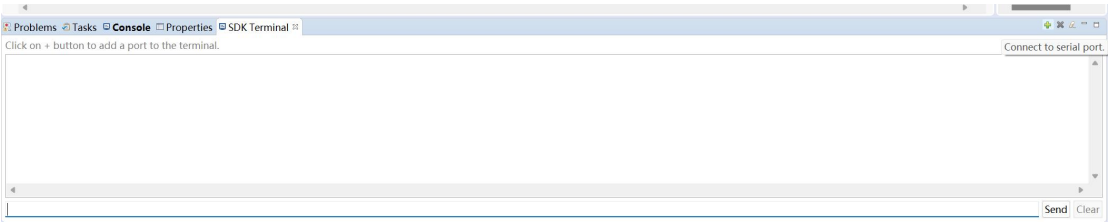
4. 跳出以下界面后，点击 Run。



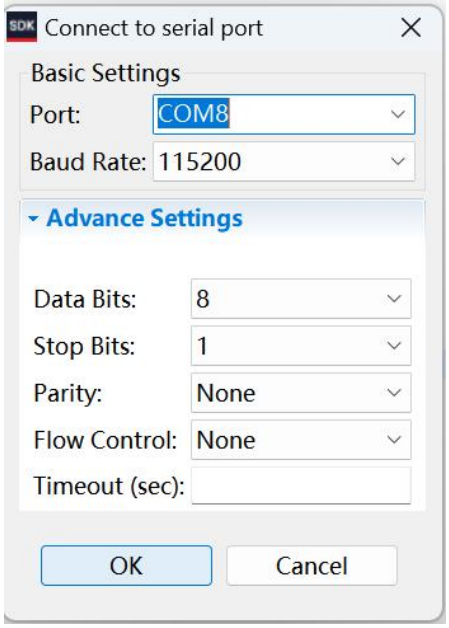
5. 查看日志，发现不报错，运行正常。



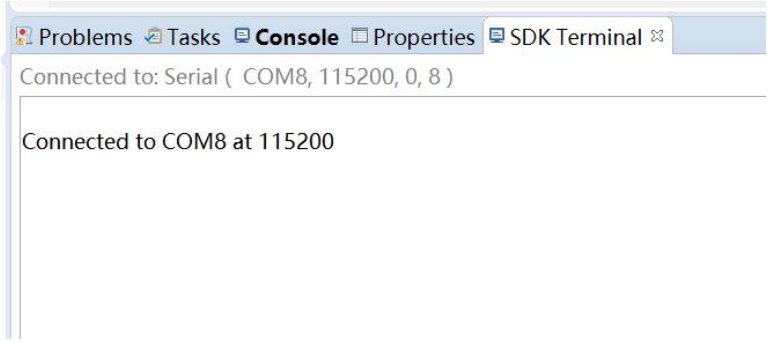
6. 点击右上角加号。



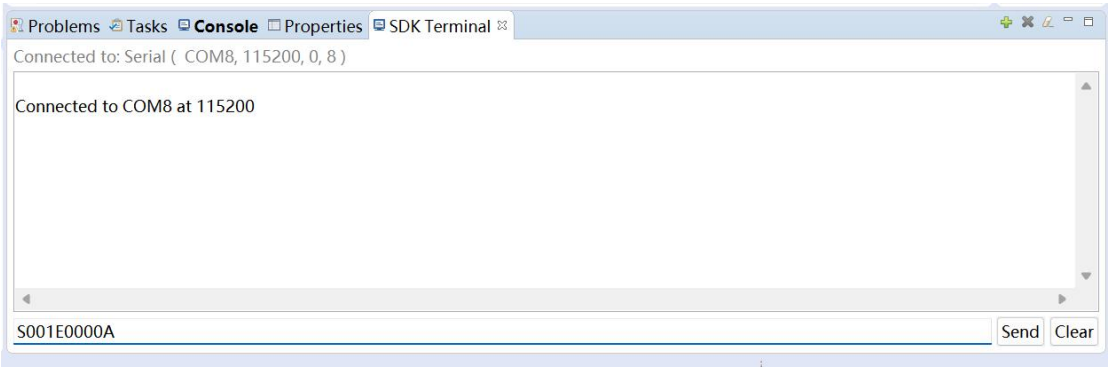
7. 跳出以下界面后，点击 OK。

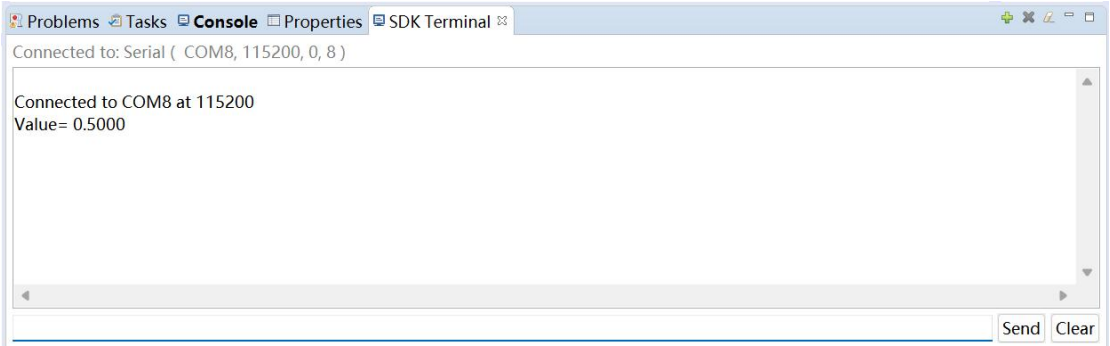


8. 连接完成。

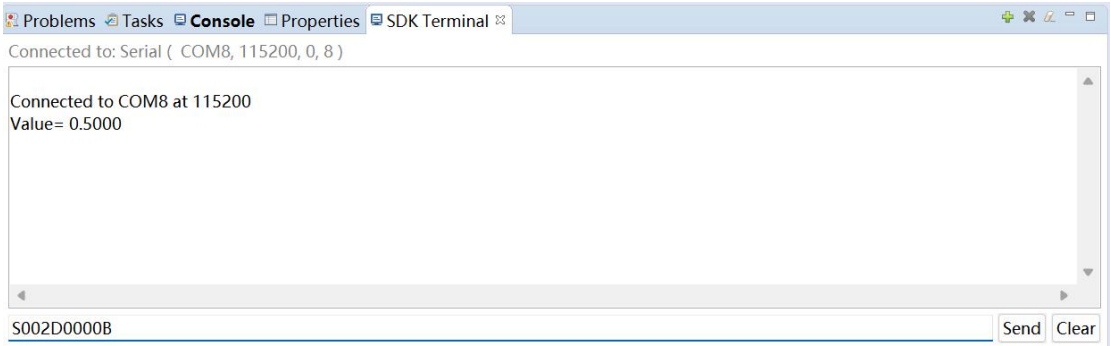


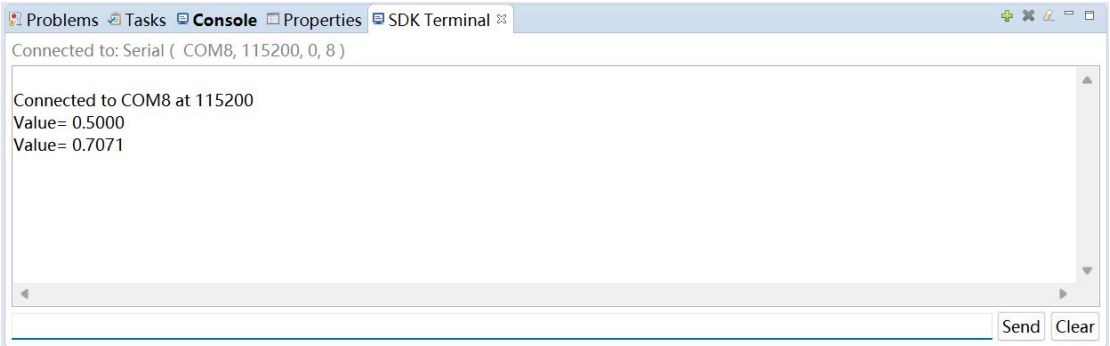
9. 以 sin30° 为例进行演示。





10. 以 cos45° 为例进行演示。





6.2 测试结果及评价

通过输入不同函数值，得到结果如下表。其中所有函数计算误差都控制在0.05%以内，证明了算法的优越性。

三角函数	理论值	实际值	误差
sin30°	0.5000	0.5000	<0.05%
cos45°	0.7071	0.7071	<0.05%
arcsin(√3/2)	60	59.9971	<0.05%
arctan(√3/3)	30	29.9854	<0.05%
arcsin(-√3/2)	-60	-59.9971	<0.05%
cos - 45°	0.7071	0.7071	<0.05%
arctan(-√3/3）	-30	-29.9854	<0.05%

7.项目中的困难与解决方案

7.1 UART 串行通信的帧同步与数据解析问题

困难描述：

由于系统采用 AXI Uartlite 进行通信，数据是以串行字节流的形式到达 PL 端的，而 Cordic 核心计算需要完整的 32 位并行数据。在早期的状态机设计中，缺乏有效的帧头检测机制，导致当串口发送速率与 FPGA 系统时钟不匹配时，接收逻辑容易出现帧错位，导致 IP 核进入错误的计算模式，输出乱码结果。

解决方案：

我们对接收模块的有限状态机进行了重构，引入了严格的帧同步校验机制。首先，我们在 IDLE 状态增加了起始位检测逻辑，仅当检测到特定的帧头字符 S 时，状态机才会被激活，从而有效过滤掉总线空闲时的噪声干扰。其次，在数据接收状态中，我们引入了计数器同步机制，强制状态机必须接收满 8 个十六进制字符后，才允许跳转至指令解析状态。最后，我们将 Uartlite 的 rx_valid 信号作为状态跳转的必要握手条件，确保了每一次移位操作都在数据总线稳定有效时进行。经过这些改进，通信接口的鲁棒性得到了显著提升，解决了数据丢包与错位问题。

7.2 硬件逻辑无法直接存储与计算浮点小数

困难描述：

在进行正弦（sin）、余弦（cos）等三角函数计算时，其输入值和输出结果绝大多数都是介于-1 到 1 之间的小数 。然而，FPGA 底层的 Verilog 寄存器本质上只能存储二进制整数（0 和 1），无法直接处理带小数点的浮点数。如果不做处理直接输入，小数部分会被截断为 0，导致硬件计算结果始终为 0 或出现严重阶梯状误差，根本无法还原真实的三角函数曲线。

解决方案：

在 PS 端输入数据前，将所有浮点数乘以 2^{16}，将其转换为 32 位整型数据再传入 PL 端。在 Cordic 核心算法中，所有的移位和加减运算均基于这个放大后的整数进行，这样硬件无需关心小数点位置，这大大简化了电路设计。当 PL 端计

算完成后，将结果传回 PS 端，再除以 2^{16} 并强制转换为浮点类型，从而还原出具有高精度的小数结果。

7.3 硬件调试的“黑盒”现象

困难描述：

在调试过程中，我们发现 PS 端程序在等待 PL 端计算完成时，经常会陷入死循环，无法获取结果。由于 FPGA 运行时其内部逻辑如同一个“黑盒”，我们无法通过简单的串口打印来观察 IP 核内部微秒级的状态跳转。我们无法确定究竟是状态机卡死在了某个中间状态，还是计数器计数逻辑错误等其他原因。

解决方案：

反复调试代码和整体逻辑，采用“假设-验证-排除”的策略逐个排查问题，上网寻找相关资料以寻求解决办法。

8. 项目的改进与创新

8.1 采用基于资源复用的通用 Cordic 统一计算引擎

在我们早期的设计方案中，为了涵盖 sin/cos 等不同函数，需要针对每种功能单独编写 Verilog 模块，这不仅导致代码冗余，更造成了 FPGA 片上逻辑资源的极大浪费。为此，我们深入分析了不同旋转模式在数学方程上的共性，设计了一个统一的数据通路。通过引入模式控制信号与参数选择信号，我们在同一套加法器和移位寄存器硬件上，成功实现了对全部 4 种基础函数的支持。这种通用架构设计显著提升了硬件复用率与系统集成度。

8.2 采用软硬协同与算法适配的计算策略

针对纯硬件环境下处理浮点交互困难、资源消耗大的问题，我们采取了 PS 端负责控制与预处理，PL 端负责核心计算的软硬协同策略。我们将逻辑复杂但计算量小的任务交由 PS 端执行，主要包括用户指令的解析以及浮点数与定点数（Q16.16/Q14）的双向转换。这充分利用了 CPU 处理复杂逻辑的灵活性，使得 PL 端无需消耗大量 LUT 资源去实现浮点运算单元。我们将计算密集型、重复性高的任务交由 PL 端（FPGA）执行，利用其并行性实现 16 级流水线的 Cordic 迭代。特别是在 arcsin 函数的实现上，我们充分利用了 Cordic 算法在向量模式下的固有特性：不同于基于数学解析式的传统实现方法，本设计通过判断 Y 分量与

输入目标值的大小关系来动态调整旋转方向，在迭代中直接锁定目标角度。这一策略巧妙地规避了复杂的硬件开方运算，在确保计算精度的同时，极大地降低了逻辑资源的占用与时序开销。

8.3 全流程高精度定点数处理

针对 FPGA 底层逻辑单元无法直接存储浮点数，且硬件浮点运算单元资源消耗过大的问题，本项目摒弃了传统的低精度近似处理，在 PS 端数据下行前采用逻辑左移 16 位（乘 2^{16}）的方式进行统一量化，将浮点域映射为 32 位定点整数域。在 PL 端核心运算过程中，所有迭代与移位操作均严格保持该精度标准，有效避免了级联运算中的精度丢失。最终在数据上行回传 PS 端时进行反量化处理。经实测，该方案在较低的硬件开销下，将所有要求的函数的计算误差成功控制在 0.05%以内，实现了硬件效率与计算精度的最佳平衡。

9. 项目总结

本次项目基于 ZYNQ 异构平台，我们成功使用 Verilog 硬件描述语言实现了底层的 Cordic 核心算法，并构建了完整的系统架构。设计中重点攻克了 PS 与 PL 端的数据交互难题，搭建了一条高效的软硬协同计算链路：用户在 PS 端通过串口下发指令帧，经由 AXI 总线转化为串行比特流送入 PL 端，PL 端利用 16 级流水线架构完成硬件加速计算后，将结果实时回传至 PS 端进行后处理与显示，实现了高效的闭环验证。

在圆满完成题目要求的 sin、cos、arcsin 及 arctan 这 4 个核心函数的前提下，我们并未盲目追求函数的数量堆叠，而是深入挖掘了算法在资源受限环境下的优化潜力。通过纯移位与加法逻辑的精简设计，我们极大地降低了硬件成本。特别是针对 arcsin 函数，我们摒弃了复杂的通用双曲模式，创新性地提出了基于数值逼近的特殊算法，有效规避了硬件开方运算的高消耗难题，在保证 Q16.16 定点精度的同时，显著优化了时序性能。

经过系统性的板级验证与多组数据测试，数据结果显示大部分运算误差极小，系统运行稳定，基本符合预期设计目标。此外，本项目目前在软硬协同的运算方法优化以及 IP 核的资源复用架构等方面仍具备不少改进空间，未来可在此基础上做出进一步的拓展与创新。

10. 参考文献

[1]刘庆,陈应兵,李智,等.基于Cordic算法的三角函数计算及改进[J].电子产品可靠性与环境试验,2025,43(05):68-74.
[2]邓强.利用Cordic算法计算平方根及其FPGA实现[J].通信技术,2013,46(07):129-131.
[3]黄海.基于Cordic的离散三角变换快速算法及其实现研究[D].哈尔滨工业大学,2014.
[4]梁杰,孙未.Cordic算法的优化及硬件实现[J].电子世界,2013,(10):85.

附录

本项目用到了如下软件和工具：

软件工具名称和版本	用途
Vivado 2018.3	用于编写底层 Cordic 代码、IP 核创建、整体模型的搭建、PS 端设计、驱动底层逻辑并实现串口数据的交互
Google Chrome 浏览器	用于辅助开发过程中的技术资料检索与问题排查